Intraday Trading CUDA Optimized

A Project Component for

Parallel and Distributing Computing (UCS645)

By

Name: Dhriti Jindal

Roll Number:102303620

Submitted to: Dr. Saif Nalband

(Assistant Professor, DCSE)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY (DEEMED TO BE UNIVERSITY)

PATIALA - 147004

MAY, 2026

Table of Contents

List of Tables

INTRODUCTION

1.1 Background

By automating decision-making processes, algorithmic trading has become incredibly popular in recent years, transforming the financial markets. The efficiency and precision of trading have been greatly improved by the use of cutting-edge computational techniques, such as parallel computing, artificial intelligence, and machine learning. The use of technical indicators to inform stock trading decisions is one such area where processing capacity is essential.

Traders employ technical indicators, which are mathematical computations based on past price, volume, or open interest data, to forecast future price changes. Bollinger bands, RSI (Relative Strength Index), MACD (Moving Average Convergence Divergence), and Moving Averages (SMA, EMA) are a few examples of these. Traders try to find possible buying or selling points for a particular asset by examining these indications.

Nevertheless, the conventional method of computing these indicators is computationally costly, particularly when dealing with big datasets or high-frequency data. More effective algorithms that can process enormous volumes of data in real time are therefore required.
By utilizing the processing capacity of GPUs, CUDA (Compute Unified Device Architecture), a parallel computing platform and programming style created by NVIDIA, enables notable acceleration in such computations. The goal of this project is to exploit CUDA's capability to develop a stock trading model that facilitates real-time trading by quickly calculating technical indicators and making buy/sell decisions.

1.2 Introduction to Problem Statement

Accuracy and quickness are essential in the financial markets. To obtain a competitive advantage, traders and institutions depend on quick data analysis and decision-making. The computational burden of evaluating massive amounts of stock data in real-time has increased dramatically as more traders use algorithmic techniques. Conventional CPU-bound trading systems are turning into a bottleneck, particularly when handling high-frequency data or when making decisions based on several technical indications. Additionally, complicated indicators like MACD, RSI, and Bollinger Bands must be calculated, frequently over sliding time periods, as part of technical analysis, which is the foundation of many trading techniques.

Although these computations are naturally parallelizable, CPU-based computers usually carry them out sequentially. Such trading models' scalability and responsiveness are constrained by this inefficiency.

The core problem this project aims to tackle is the **inefficiency** of traditional technical analysis-based trading systems in processing data fast enough to respond to market move-ments in real time. By shifting computationally intensive parts of the trading system, especially indicator calculations and trade signal generationˆato the GPU using CUDA, we can exploit parallelism and achieve significant performance gains.

The following research and implementation questions are the focus of this project:

- How can technical indicator computations be accelerated with CUDA?
- In comparison to a CPU-only approach, what performance gains are possible?
- What effects does this acceleration have on trading decisions' efficacy and responsiveness?

By doing this, the project hopes to create a single-stock trading model with excellent performance that incorporates:

- GPU-powered technical analysis
- Ingestion of data in real time
- Useful buy/sell indicators

Despite being concentrated on a single asset, this version incorporates portfolio actions, technical indicators, and candlestick chart graphical visualization.

LITERATURE REVIEW

Algorithmic trading systems have advanced significantly as a result of the confluence of technology and finance. These systems conduct trades automatically using pre-established rules based on statistical models, machine learning, or technical indicators. High-performance computing (HPC) techniques, especially GPU-based parallel computing, are essential to increasing their responsiveness and efficiency.

2.1 Technical Indicators in Trading Systems

Understanding market patterns and spotting trading opportunities have traditionally been accomplished through the use of technical analysis. In a variety of market situations, indicators like as the Simple Moving Average (SMA), Exponential Moving Average (EMA), Relative Strength Index (RSI), MACD, and Bollinger Bands have shown efficacy [1]. These indicators require a lot of processing power, particularly when using large-scale or high-frequency historical data. The mathematical foundation and empirical significance of technical indicators have been extensively discussed in earlier works. More recent research investigates how trading accuracy can be increased and false signals can be decreased by integrating numerous indicators.

2.2 Limitations of Traditional CPU-Based Systems

Conventional CPU-based trading models frequently have speed and scalability issues. Sequential processing becomes a bottleneck as datasets get bigger and more complicated, particularly in real-time trading. The inefficiency of employing CPUs for real-time financial data analysis has been noted by a number of studies, especially in high-frequency trading situations

2.3 GPU and CUDA in Financial Computing

The use of GPU acceleration in financial computing has gained popularity, especially as contemporary trading systems require quicker and more effective data processing. Because of their sequential execution approach, traditional CPU-based systems frequently find it difficult to meet real-time requirements due to the rapid increase in data volume and complexity. GPUs, on the other hand, are made for parallel computing, which enables the completion of several calculations at once. This makes them ideal for financial applications like market data analysis and technical indicator evaluation that require frequent numerical computations. Systems can greatly cut calculation time, increase scalability, and improve the overall performance of algorithmic trading models operating under time-sensitive scenarios by utilizing GPU architectures.

CUDA-based financial algorithm implementations showed notable performance gains in a comparative analysis by Matthew Dixon et al. (2013) [5], outperforming conventional CPU approaches by up to 100 times. These findings demonstrate the great potential of GPU-accelerated systems to offer real-time responsiveness and high-throughput processing in trading environments.

2.4 Visualization in Trading Systems

Even while visualization tools like Matplotlib and Plotly are frequently used to analyze trade performance and comprehend market activity, it is still difficult to integrate them seamlessly into real-time trading systems. Visualization, which uses charts and graphical representations to communicate complex financial data, is crucial to its simplification. To decipher trends and spot possible buy or sell signals, traders rely on visual components including moving averages, indicator overlays, and candlestick charts. By offering clarity that raw numerical data frequently lacks, these graphical representations enhance decision-making. Despite their benefits, latency and processing limitations make it challenging to integrate these tools into high-speed trading systems, particularly when handling massive amounts of streaming financial data.

In order to overcome this constraint, this project employs visualization through Python-based plotting techniques, producing portfolio performance plots, indicator graphs, and candlestick charts as PNG outputs. This allows for clear post-analysis of trading strategies and overall system behavior without compromising execution speed

RESEARCH GAPS

Despite the increasing integration of computational methodologies in finance, there remain several significant research gaps in the area of technical analysis, high-performance computing, and real-time trading systems. Despite being strong in fundamental theory and simulation studies, the present literature reveals limitations in actual implementation, especially for GPU-accelerated trade logic. These shortcomings serve as the foundation for the current project's objective and course.s.

3.1 Lack of Real-Time GPU-Accelerated Technical Indicator Systems

There is a conspicuous lack of research that directly applies GPU acceleration to technical indicator-based trading systems, despite the fact that CUDA and GPU computing have been successfully applied to financial simulations and option pricing models. Rather than real-time trade decision-making in volatile markets, the majority of current works concentrate on theoretical performance benchmarks or batch simulations.

3.2 Integration Challenges in End-to-End Trading Pipelines

Studies rarely take into account a complete pipeline, which includes data fetching, preprocessing, indicator calculation, and trade signal execution. Instead, they frequently isolate performance gains in either data processing or model computation. Research on closely integrating GPU-accelerated components into a modular, real-time trading platform is scarce.

3.3 Lack of Evaluation on Lightweight, Resource-Constrained Systems

Many implementations assume access to high-end servers or multi-GPU clusters. Few works explore how much performance gain can be achieved using CUDA on consumer-grade GPUs, making them less applicable to individual traders or small-scale setups.

3.4 Visualization Often Treated as an Afterthought

While some literature acknowledges the importance of data visualization for human decision-making, it is often treated as a separate task rather than being integrated into the system pipeline. Real-time systems that support visualization of both indicators and trading decisions are still underexplored ˆespecially those that interact with GPU-processed data efficiently.

3.5 Multi-Asset Scalability and Optimization Gaps

Most academic studies and small-scale systems focus on a single asset or ticker, with limited provisions for extending to a multi-asset portfolio analysis. Optimizing technical analysis and trading decisions concurrently across multiple stocksˆ especially using GPU resources efficiently is still an open research problem.

PROBLEM FORMULATION

In the realm of algorithmic trading, technical indicators are extensively used to guide buy and sell decisions. These indicators often rely on historical price data and are computed over sliding windows, resulting in high computational overhead. When multiple indicators are used concurrently as is common in robust trading strategies the processing time increases significantly, especially as the volume of stock data grows. This latency poses a major problem in environments where real-time decision-making is crucial, such as intraday or high-frequency trading.

Most conventional trading systems are implemented using CPU-based architectures, which process indicator computations sequentially. While these systems can be accurate, they struggle with speed and scalability. As a result, traders relying on these systems may miss time-sensitive opportunities or suffer from lag in execution, particularly in volatile market conditions.

At the same time, advancements in GPU programming, particularly through NVIDIA CUDA framework, offer a promising solution. GPU architectures are well-suited for parallel computation and can execute thousands of threads simultaneously, making them ideal for the repetitive, data-parallel operations inherent in technical indicator calculations.

This project addresses the problem of slow, sequential indicator processing in traditional trading systems by formulating a CUDA-accelerated trading model. The model focuses on offloading computationally intensive indicator calculations such as SMA, EMA, RSI, MACD, and Bollinger Bands to the GPU for parallel execution. It uses pre-existing historical stock data stored in CSV format, parses it, computes indicators in parallel, and makes trading decisions based on predefined logic.

By accelerating these operations using CUDA, the model aims to significantly reduce processing latency, enabling faster, more responsive trading. Although the current imple-mentation is focused on a single asset and is focused on a single asset and includes visualization of trading outputs via Python-based charts, it lays the groundwork for a scalable and performance-oriented trading engine suitable for real-world deployment.

Objectives

The main objective of this project is to design and implement a high-performance, GPU-accelerated trading system that leverages technical indicators for real-time stock trading decisions.

5.1 Primary Objectives

- To develop a CUDA-based computation engine for technical indicators such as SMA, EMA, RSI, MACD, and Bollinger Bands, optimizing them for parallel execution on the GPU.
- To build an end-to-end stock trading model that integrates real-time data fetching, indicator computation, trade decision logic, and result logging.
- To minimize latency in decision-making, enabling more responsive trading actions compared to traditional CPU-bound systems.
- To evaluate performance improvements in indicator computation between GPU (CUDA) and CPU implementations.

5.2 Secondary Objectives

- To implement a robust trade signal generation mechanism based on multiple technical indicators.
- To maintain a structured portfolio log capturing trade actions, execution prices, and value over time via CSV for later evaluation.
- To assess the resource efficiency of running the model on consumer-grade GPUs, enhancing accessibility for small-scale or individual traders.

Methodology

—

This project implements a CUDA-accelerated stock trading system in C++, integrat-ing real-time data processing, technical indicator computation, and trading logic. The methodology involves building an efficient pipeline that shifts computationally expensive operations to the GPU using CUDA, while leveraging CPU resources for I/O and logic orchestration.

6.1 System Architecture Overview

The system is composed of the following modules:

6.1.1 Data Acquisition Module: Loads pre-existing daily TSLA stock price data from CSV files stored in the repository.

6.1.2 Preprocessing Module: Cleans, parses, and structures the raw data into arrays for indicator calculation.

6.1.3 CUDA Indicator Kernel Module: Implements GPU-parallelized computation of technical indicators (SMA, EMA, RSI, MACD, Bollinger Bands).

6.1.4 Trading Logic Module: Applies decision rules based on indicator thresholds to gen-erate buy/sell signals.

6.1.5 Portfolio Management Module: Records trade actions and portfolio value over time into CSV logs.

6.2 Workflow Pipeline

1. Fetch Stock Data: Use the Alpha Vantage API to retrieve past OHLC (Open, High, Low, Close) data. Save in CSV format.
2. Read and parse the CSV file to preprocess the data. Transform information into numerical arrays that are appropriate for GPU computation, such as float arrays.
3. Preprocess Data: Read and parse the CSV. Convert it to suitable numerical arrays (e.g., float arrays) for GPU processing.
4. GPU Indicator Computation: Start the CUDA kernels to compute:

- Simple Moving Average, or SMA

- Exponential Moving Average, or EMA

- Relative Strength Index, or RSI

- Bollinger Bands

- The Stochastic Oscillator

- Moving Average Convergence Divergence, or MACD

- ATR

5. Trade Signal Generation: Create signals based on user-specified logic (e.g., MACD crossover = Buy/Sell, RSI § 30 = Buy). After gathering computed indicators from the GPU, the CPU handles this phase.

6.3 Tools & Libraries

- C++: Core programming language for the trading logic and data handling.
- CUDA: NVIDIA's parallel computing platform for accelerating indicator computation.
- CSV & Json Parsing Libraries: For efficient reading and writing of trade logs and price data. (nlohmann)

Algorithm 1 GPU-Accelerated Exponential Moving Average (EMA) Computation

```
function computeEMA_GPU(prices, period)
    n ← length of prices
    if n < period then
        Display warning and return
    end if
    multiplier ← 2 / (period + 1)
    Allocate GPU memory: d_prices, d_output
    Copy prices to d_prices
    Launch emaKernel(d_prices, d_output, n, period, multiplier)
    Copy d_output back to host as result
    Free GPU memory
end function

function emaKernel(prices, output, n, period, multiplier)
    if thread ID is 0 and block ID is 0 then
        Compute initial SMA over first period elements
        output[0] ← initial SMA
        for i = period to n − 1 do
            output[i − period + 1] ← (prices[i] − output[i − period]) · multiplier
  + output[i − period]
        end for
    end if
end function
```

Algorithm 2 CUDA Kernel for Stochastic Oscillator (%K and %D) Computation

```
function STOCHASTICKERNEL(close, high, low, kOut, dOut, n, periodK, smoothK, periodD)
    if thread ID ≠ 0 or block ID ≠ 0 then
        return
    end if
    rawKSize ← n − periodK + 1                                  ▷ Step 1: Compute Raw %K
    for i = 0 to rawKSize − 1 do
        highest ← max of high[i . . . i + periodK − 1]
        lowest ← min of low[i . . . i + periodK − 1]
        if highest = lowest then
            rawK[i] ← 50.0
        else
            rawK[i] ← 100 · (highest − lowest) / (close[i + periodK − 1] − lowest)
        end if
    end for
                                                                ▷ Step 2: Smooth %K (moving average)
    smoothKSize ← rawKSize − smoothK + 1
    for i = 0 to smoothKSize − 1 do
        kOut[i] ← average of rawK[i . . . i + smoothK − 1]
    end for
                                                                ▷ Step 3: Compute %D (SMA of smoothed %K)
    dSize ← smoothKSize − periodD + 1
    for i = 0 to dSize − 1 do
        dOut[i] ← average of kOut[i . . . i + periodD − 1]
    end for
end function
```

DATASETS

Table 2: Stock Price Dataset Used for Real-Time Technical Indicator Computation

Name of Datasets	**Different Features**	**Size**	**Year**	**Organization**
TSLA Daily Stock Price Data	Open, High, Low, Close, Volume One entry per trading day	~100 rows	2024-2025	Pre-existing CSV in repository

RESULTS AND DISCUSSION

Pre-existing daily TSLA stock data loaded from repository CSV files and analyzed on an NVIDIA T4 GPU (Google Colab environment) was used to evaluate the CUDA-accelerated trading model. The outcomes show that GPU acceleration is feasible and improves performance in real-time technical analysis.

8.1 Indicator Computation Performance

Significant performance gains were observed in benchmark comparisons between CPU and GPU computations of indicators:

Table 3: Stock Price Dataset Used for Real-Time Technical Indicator Computation

Simple Moving Average (SMA)	3.3656 ms	0.5535ms	6x
Exponential Moving Average (EMA)	2.8091 ms	0.4576ms	6x
Relative Strength Index (RSI)	6.4405 ms	1.1934ms	5.3x
Bollinger Bands	10.4532 ms	3.2598ms	3x
MACD	11.0155 ms	4.6283	2.5x
Stochastic Oscillator	12.7368 ms	3.2894	4x
Average True Range (ATR)	5.7527 ms	2.3421	2.5x
Overall	50.7527 ms	15.3421	3.3x

These results indicate that technical indicators, particularly those based on rolling windows, are highly parallelizable and benefit substantially from GPU execution.

8.2 Trading Logic Execution

The model effectively produced buy/sell signals based on predetermined circumstances (e.g., RSI § 30 triggers Buy, MACD crossing triggers Sell). Every action, execution price, and changed portfolio value were recorded in a CSV file.

The logic architecture is modular and intended to accommodate numerous stock tickers with small extensions, even though it is now restricted to a single asset.

8.3 Discussion

This project confirms that the speed of technical indicator computations is greatly increased by GPU acceleration via CUDA. Although real-time accuracy and performance seem encouraging, expanding the system to include error handling, real broker integration, and visualization would be crucial future stages. With these improvements, this model might develop into a scalable, lightweight trading assistant for institutional or retail use.

References

[1] J. J. Murphy, *Technical Analysis of the Financial Markets*.New York Institute of Finance, 1999.

[2] F. Rundo, "A survey on technical indicators in algorithmic trading," *International Journal of Computer Applications*, 2019

[3] S. B. Achelis, *Technical Analysis from A to Z*.I. Aldridge, *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*. Wiley, 2013.

[4] D. Shah, A. Braverman, and J. Tsitsiklis, "Visual analytics in financial modeling," *IEEE Transactions on Visualization and Computer Graphics*, 2018.

[5] R. Rakhsha, T. T. Nguyen, and J. Wang, "Cuda-based high-performance option pric-ing," *Journal of Computational Finance*, 2017.

[7] M. Dixon, I. Halperin, and P. Bilokon, *Machine Learning in Finance: From Theory to Practice*. Wiley, 2013.